System Design

Specification (SDS)

Project: Taste Matcher
Author: Srirama Prabhav R.
Date: May 5, 2026

1. Architectural Overview

Taste Matcher is built on a local-first, lightweight architecture to maximize speed and eliminate external dependencies. The system runs entirely on the user's machine as a Node.js Express server, optionally wrapped in an Electron desktop shell.

Layer	Technology	Role
Backend Runtime	Node.js 18+	Application server, scoring engine, file I/O
HTTP Framework	Express.js 4.x	REST API routing and static file serving
HTTP Client	Axios	TMDb API requests with error handling
CSV Parsing	csv-parse 6.x	Letterboxd CSV ingestion with auto header detection
External Data	TMDb REST API v3	Movie/TV metadata: genres, credits, keywords, collections
Frontend	Vanilla HTML / CSS / JS	Framework-free SPA — no React, no Vue, no bundle step
State Persistence	Flat JSON files (/cache/)	No database — instant reads, fully portable
Desktop Wrapper	Electron 28	Cross-platform native app; IPC bridge for file operations
Environment Config	dotenv (.env)	API key and port management in plain-node mode

2. Data Flow

2.1 Startup Sequence

1. loadMovieCacheFromDisk() — deserializes tmdb_cache.json into the in-memory movieCache Map
2. loadDerivedCacheFromDisk() — loads pre-computed taste model, recommendations, overlap details, rewatch ranking
3. loadHiddenRewatches() — loads hidden_rewatches.json into a Set
4. loadUserWeightsFromDisk() — loads user_weights.json; overrides DEFAULT_WEIGHTS if present
5. loadAllData() — attempts to restore from data_state.json; falls back to CSV parse if absent

2.2 Request Pipeline — Ranked Watchlist

6. GET /api/recommendations received
7. buildTasteModel() — iterates all rated films, fetches TMDb metadata for each (cache-first), builds nine Bayesian-smoothed dimension profiles and ratedItemsMeta array for k-NN
8. For each watchlist item: getMovieDetailsForItem() (cache-first), then scoreItem(movie, taste)
9. scoreItem() computes nine dimension affinity scores, runs neighborComponent() for k-NN similarity, applies activeWeights, returns predictedScore
10. Results sorted descending by predictedScore, cached in cachedRecommendations
11. JSON response sent to frontend; UI constructs card grid from response

3. Module Design

3.1 scoreItem(movie, taste) — Core Scoring Function

The central algorithm. Takes a TMDb movie object and the current taste model, returns a composite affinity score and per-dimension breakdown.

Dimension	Weight	Computation
Keyword Affinity (kwAff)	0.22 (default)	affinityFromProfile(keywords, keywordProfile, keywordStats, 3) — Bayesian smoothed mean rating across matching keywords
Genre Affinity (genreAff)	0.18 (default)	affinityFromProfile(genreIds, genreProfile, genreStats, 3)
TMDb Community Score (tmdbNorm)	0.15 (default)	normalizeTmdbRating(vote_average) — linear normalize to [0,1]
Director Affinity (dirAff)	0.10 (default)	affinityFromProfile(directors, directorProfile, directorStats, 3)
Neighbour Similarity (neighbor)	0.10 (default)	neighborComponent() — k-NN Jaccard over ratedItemsMeta
Writer Affinity (writerAff)	0.08 (default)	affinityFromProfile(writers, writerProfile, writerStats, 3)
Geography Affinity (geoAff)	0.07 (default)	Mean of countryAff and regionAff
Decade Affinity (decadeAff)	0.05 (default)	affinityFromProfile([decade], decadeProfile, decadeStats, 4)
Collection Affinity (collAff)	0.05 (default)	affinityFromProfile([collectionId], collectionProfile, collectionStats, 2)

All weights are read from activeWeights at call time. When the user saves a priority ranking, weightsFromPriorityRanking() converts it to geometric-decay weights (each rank = 0.75× the previous), normalized to sum 1.00.

3.2 neighborComponent(candMeta, ratedMeta) — k-NN Layer

Computes structural similarity between a candidate film and the user's entire rated history:

- filmSim() computes: 0.50 × moodSim + 0.20 × regionJaccard + 0.15 × eraSim + 0.15 × directorMatch
- moodSim = 0.65 × genreJaccard + 0.35 × keywordJaccard
- eraSim: 1.0 if same decade, 0.5 if ±10 years, 0.0 otherwise
- Neighbours with similarity ≤ 0.12 are discarded; top 30 retained
- Final score: Σ(sim × rating) / Σ(sim) × confidence, where confidence = min(1, neighbours/8)

3.3 buildTasteModel() — Taste Profile Constructor

Iterates the full ratings list. For each rated film, fetches TMDb metadata and accumulates stats across all nine dimension maps. Applies Bayesian smoothing with SMOOTH_K = 5:

```
smoothedScore(key) = (totalRating + globalMean × 5) / (count + 5)
```

This pulls low-sample entries toward the user's global mean, preventing a single 5-star film from dominating a dimension.

3.4 Overlap Detection — computeOverlapAndClean()

Dual-strategy matching to identify films present in both ratings and watchlist:

- Primary: normalizeUrl(url) exact match — handles the common case where URLs are canonical
- Fallback: titleKey(title, year) — catches edge cases where URL formats differ across export versions

Detected overlap items are removed from the watchlist ranking queue and placed in the rewatch pool.

3.5 Rewatch Ranking Formula

```
rewatchScore = userRatingNorm × (0.70 + 0.30 × modelScore)
```

Design intent: the user's own rating is the anchor. The model can nudge a film up or down by ±30%, but cannot override explicit user judgment. A 2-star film the model loves can score at most ~0.50 — correctly below any 3-star rated film.

4. Persistence Layer

File	Contents	Updated When
tmdb_cache.json	Map of all fetched TMDb metadata. Key: 'id_{tmdbId}' or '{title}_{year}'	On every new TMDb API call
derived_cache.json	cachedTasteModel, cachedRecommendations, cachedOverlapDetails, cachedRewatchRanking	After any build/calculate function completes or is invalidated
data_state.json	ratings[], watchlist[], overlapItems[] arrays	After every mutation (add-rating, mark-watched, remove, etc.)
user_weights.json	activeWeights object (9 dimension keys → weight values)	After POST /api/user-weights or /api/user-weights/reset

File	Contents	Updated When
hidden_rewatches.json	Set of normalized Letterboxd URLs for hidden rewatches	After POST /api/hide-rewatch

5. API Endpoints

Method	Endpoint	Description
GET	/api/recommendations	Returns ranked watchlist. Triggers buildTasteModel() and calculateRecommendations() if cache is cold.
GET	/api/rewatch-ranking	Returns rated films that are also on the watchlist, ranked by rewatchScore.
GET	/api/overlap	Returns overlap details with TMDb metadata for each rewatch candidate.
GET	/api/genre-profile	Returns per-genre affinity data for the genre profile view.
GET	/api/ratings	Returns raw ratings array.
GET	/api/watchlist	Returns raw watchlist array.
GET	/api/search-ratings	Full-text search across rated films. ?all=true returns full list.
GET	/api/export-ranked-watchlist	Streams Letterboxd v7 CSV. ?includeRewatches=true appends rewatch pool.
GET	/api/app-status	Returns counts for the settings drawer stats panel.
GET	/api/user-weights	Returns current activeWeights and DEFAULT_WEIGHTS.
POST	/api/user-weights	Accepts ranked dimension array; computes and saves geometric-decay weights.
POST	/api/user-weights/reset	Resets activeWeights to DEFAULT_WEIGHTS.
POST	/api/add-rating	Adds or updates a rating. Triple duplicate check. Invalidates all caches.
POST	/api/update-rating	Updates an existing rating by URL or title+year. Invalidates all caches.
POST	/api/add-to-watchlist	Adds film to watchlist. Triple duplicate check; moves to rewatches if already rated.
POST	/api/mark-watched	Moves film from watchlist to ratings with a given star rating.
POST	/api/remove-from-watchlist	Removes a film from the watchlist.

Method	Endpoint	Description
POST	/api/hide-rewatch	Hides a film from the rewatch ranking. Persisted to hidden_rewatches.json.
POST	/api/reload-watchlist	Hot-reloads watchlist.csv without touching ratings or TMDb cache.
POST	/api/invalidate-recommendations	Clears recommendation caches without a full state reset.
GET	/api/validate-tmdb-key	Tests the configured TMDb API key against a known endpoint.
GET	/api/failed-items	Returns list of films that could not be resolved on TMDb.

6. Known Constraints & Technical Debt

6.1 Synchronous API Bottleneck

TMDb metadata is fetched sequentially inside a for...of loop in buildTasteModel() and calculateRecommendations(). For large libraries (500+ films), the first-run fetch phase can take several minutes.

Proposed v3.0 solution: Replace with Promise.all() batching regulated by a p-limit concurrency limiter. No cache migration required; this is a pure performance improvement.

6.2 Linear Scoring Weights

The nine dimension weights are static constants (DEFAULT_WEIGHTS). The user-priority system introduced in v3.1 allows reordering but not arbitrary numeric fine-tuning within a tier.

Proposed next step: Per-dimension numeric sliders for power users who want granularity beyond rank order.

6.3 No Real-Time Letterboxd Sync

The system operates on static CSV exports. New Letterboxd ratings are not reflected until the user re-exports and reloads, or uses the manual add-rating endpoint.

Proposed exploratory: Letterboxd OAuth live sync, contingent on Letterboxd opening a public OAuth API.